Gestion des erreurs en C système

October 30, 2025

> Cette documentation doit être utilisée pour gérer correctement les erreurs dans les différents TPs de système.

1 Le manuel linux

Le manuel linux contient la description exhaustive des fonctions de la LibC. Les appels systèmes se trouvent dans **la section 2**. Les commandes ci-dessous permettent d'utiliser le manuel :

Trouver dans quelle section se trouve une fonction:

```
$man -f open
open (2)              - open and possibly create a file
open (1)              - opens a file or URL in the user s preferred application
```

```
$man 2 open
NAME
       open, openat, creat - open and possibly create a file

SYNOPSIS
       #include <sys/types.h>
       #include <sys/stat.h>
       #include <fcntl.h>

       int open(const char *pathname, int flags);
       int open(const char *pathname, int flags, mode_t mode);
...
```

Les sections importantes pour la gestion des erreurs sont **RETURN VALUE** et **ERRORS**. Par exemple pour la fonction **open()**:

```
RETURN VALUE
open(), openat(), and creat() return the new file descriptor (a nonnegative integer), or -1
    if an error occurred (in which case, errno is set appropriately).
```

```
ERRORS
       open(), openat(), and creat() can fail with the following errors:

       EACCES The  requested  access  to the file is not allowed, or search permission is
    denied for one of the directories in the path prefix of pathname, or the file did not
    exist yet and write access to the parent directory is not allowed.  (See also
    path_resolution(7).)
       EBUSY  O_EXCL was specified in flags and pathname refers to a block device that is in
     use by the system (e.g., it is mounted).
...
```

2 Principe des codes d'erreur

Les erreurs rencontrées lors des appels système **ne provoquent pas l'arrêt immédiat de l'application** (mais souvent aboutissent quelques instructions plus tard à des segfault etc ...).

Ces erreurs sont en faite des indications destinées à l'application (sur l'état du système: mémoire, périphérique etc ...)

On peut donc soit terminer l'application en précisent les causes (nom de fichier, mémoire saturée, pid inexistant ...) ce qui est le cas de la plupart des commandes exécutées dans un terminal, soit remonter l'erreur et continuer son exécution c'est le cas des intepréteur de commande comme les shell (bash, csh, ...)

2.1 Les appels système

Les appels système sont codés de tel façon qu'ils retournent une **valeur négative en cas d'échec**. Cette valeur négative est représentative de l'erreur en question et correspond à une macro contenue dans le fichier **errno.h**.

La **LibC encapsule ces appels système dans des fonctions**. Ces fonctions placent la valeur de retour des appels dans la variable globale errno retournent -1 (ou NULL) en cas d'échec.

```
/*
    Definition de OPEN dans la LibC
*/
int open(const char *pathname, int flags, mode_t mode) {

    //Le noyau execute l appel systeme et retourne
    //le code d erreur si pb
    int ret = syscall(SYS_openat, AT_FDCWD, pathname, flags, mode);

    //Place le code d erreur dans errno et retourne -1
    //ce qui indique une erreur
    if (ret < 0) {
        errno = -ret;   //
        return -1;
    }
    return ret;
}
```

Pour obtenir La démarche consiste donc à inclure l'en-tête **errno.h** et dans le cas ou les fonctions utilisées retourne -1 traiter la valeur contenue dans errno.

2.2 Retours typiques de la LibC

Appel système	Retour en cas de succès	En cas d'erreur
`open()`	≥ 0 (descripteur de fichier)	`-1`
`read()` / `write()`	≥ 0 (nombre d'octets)	`-1`
`fork()`	> 0 (PID du fils) ou 0 (dans le fils)	`-1`
`wait()`	PID terminé	`-1`
`close()`	0	`-1`

Table 1: Valeurs de retour des principaux appels système

Selon les fonctions la valeur de retour peut varier, il est donc important de lire la définition dans le manuel.

2.3 Codes d'erreurs typiques placés dans errno

Code	Signification	Appels systèmes
EAGAIN	Ressource temporairement indisponible	`fork()`, `read()` non bloquant
ENOMEM	Mémoire insuffisante	`fork()`, `mmap()`, `brk()`
EINTR	Appel interrompu par un signal	`read()`, `wait()`, `nanosleep()`
ECHILD	Aucun processus fils existant	`wait()` sans enfant en attente
EPERM	Opération non autorisée	`kill()` vers un autre utilisateur, `setuid()`
EINVAL	Argument invalide	Paramètre incorrect dans un appel système
ESRCH	Processus inexistant	`kill()` sur un PID mort ou invalide

Table 2: Principaux codes d'erreur liés aux processus et à la gestion système

2.4 Exemple de l'appel système open

L'appel système open peut échouer pour de multiples raisons notamment: la place sur le périphérique, le nom du fichier, le chemin d'accès, les droits etc ... C'est donc un bon exemple !

Code d'erreur (`errno`)	**Signification**	**Cause typique / Contexte**
EACCES	Permission refusée	*L'utilisateur n'a pas les droits de lecture/écriture sur le fichier ou le répertoire.*
EEXIST	Fichier déjà existant	*Utilisation de `O_CREAT | O_EXCL` alors que le fichier existe déjà.*
EISDIR	Le fichier est un répertoire	*Tentative d'ouverture d'un répertoire en écriture (`O_WRONLY` ou `O_RDWR`).*
EMFILE	Trop de fichiers ouverts (processus)	*Le processus a atteint la limite locale de descripteurs de fichiers (`ulimit -n`).*
ENOENT	Fichier ou chemin inexistant	*Chemin incorrect ou répertoire parent manquant, ou `O_CREAT` non utilisé.*
ENOTDIR	Élément du chemin non répertoire	*Un composant du chemin attendu comme répertoire ne l'est pas.*
ENODEV	Aucun périphérique correspondant	*Fichier spécial associé à un périphérique inexistant.*
EROFS	Système de fichiers en lecture seule	*Tentative d'ouverture en écriture sur un système monté en lecture seule.*
EFAULT	Adresse utilisateur invalide	*Le pointeur `pathname` est hors de l'espace mémoire du processus.*
EINVAL	Argument invalide	*Flags incohérents (ex : `O_TMPFILE` sans `O_WRONLY` ou `O_RDWR`).*
EPERM	Opération non autorisée	*Restrictions de permission spéciales (ex : périphérique protégé).*
EOVERFLOW	Taille de fichier trop grande	*Le fichier dépasse les capacités du type `off_t` (en 32 bits).*

Table 3: Principales erreurs possibles de l'appel système `open()`.

Par exemple le programme suivant provoquera l'erreur **EEXIST** à la deuxième exécution.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <errno.h>
#include <string.h>
#include <unistd.h>

/* Test en appelant 2x la fonction */
void main(void){

        //comportement des flags  O_CREAT | O_EXCL | O_WRONLY
        int fd = open("data.txt", O_CREAT | O_EXCL | O_WRONLY, 0644);
        if (fd == -1) {
                if (errno == EEXIST)
                        fprintf(stderr, "Erreur : le fichier existe
                           deja !\n");
                else
                        perror("open");

        }

}
```